

Software Design Description 002

Walsh-Hadamard Transforms

Brief

Originally the plan for this device was to use FFTs (Cooley-Tukey) on an FPGA to extract coefficients/group data for an easy encoding and optimally compressed real-time pipeline. That was until I chanced upon a simple (infamous) trick when doing a DSP homework.

```
// Essentially,  
bool s = 0x0; // let this be some control variable  
y = (s) ? x : -x;
```

```
// Can be simplified to
```

```
y = (x ^ -s) + s; // By twos complement
```

Though this simplification may seem trivial, it is a foundational feature of the WHT. Furthermore, by taking out branching logic (a mux), and simplifying it in the form of 1 xor and an add, you have a much more efficient implementation in complexity, power, and time.

Mathematical Description

$$X[k] = \sum_{i=0}^{N-1} x[i] \times (-1)^{\langle k, i \rangle}$$

where $\langle k, i \rangle$ is the bitwise inner product of k and i (goes to ± 1 . Oh wow would you look at that! It's a COND_NEG!). Inversely,

$$x[i] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \times (-1)^{\langle k, i \rangle}$$

or in matrix form (which is probably more helpful for this application). Think of it as the multiplication of the Hadamard matrix.

$$H_1 = [1]$$

$$H_{2n} = \begin{bmatrix} H_n & H_n \\ H_n & -H_n \end{bmatrix}$$

From Identity to Implementation

The conditional negate identity eliminates multiplication from the WHT entirely. Each output $X[k]$ is just the sum of every input, each conditionally negated by a single bit:

```
s = parity(k & i) // which bit? AND + XOR tree  
contribution = (x[i] ^ -s) + s // the trick: +/-x with no mux  
X[k] += contribution // accumulate
```

Since every input's contribution is independent, this is embarrassingly parallel – you could compute all N contributions simultaneously with N conditional negate units feeding an adder tree. For $N=128$, that's a single-cycle transform at the cost of 128 XOR trees and a 128-input reduction adder. On a large FPGA or ASIC this is viable when latency matters more than area.

The butterfly decomposition takes the opposite trade. Adjacent rows of the Hadamard matrix differ by exactly one bit, so rather than evaluating `parity(k & i)` for every (k, i) pair independently, you can process one bit position per stage. At each stage, the two possible outcomes – “this bit was 0” and “this bit was 1” – are computed simultaneously

as $a + b$ and $a - b$, and stored for the next stage. This factors N^2 conditional negations into $N \log_2 N$ additions across $\log_2 N$ sequential stages.

For this device, area and power matter more than latency (the LoRa link is the bottleneck, not the transform), so the butterfly is the right choice – one adder, one subtractor, pure bitwise control logic, and a sequential walk through $\log_2 N$ stages. The same fundamental trick still applies, albeit implicitly.

Implementation Logic

```
def fwht(x):
    N = len(x)
    stride = 1
    while stride < N:
        for i in range(0, N, stride * 2):
            for j in range(stride):
                a, b = x[i + j], x[i + j + stride]
                x[i + j] = a + b
                x[i + j + stride] = a - b
            stride *= 2
    return x
```

which more realistically would look more like

```
class FWHT:
    def __init__(index, data):
        index.x = data[:]
        index.N = len(data)
        index.stride = 1
        index.i = 0
        index.j = 0
        index.done = False

    def tick(index):
        if index.done:
            return

        # butterfly
        a = index.x[index.i + index.j]
        b = index.x[index.i + index.j + index.stride]
        index.x[index.i + index.j] = a + b
        index.x[index.i + index.j + index.stride] = a - b

        # advance j
        index.j += 1
        if index.j >= index.stride:
            index.j = 0
            index.i += index.stride << 1

        # advance i / next stage
        if index.i >= index.N:
            index.i = 0
            index.stride <=< 1

        if index.stride >= index.N:
            index.done = True
```

but wait, it gets better

```
class FWHT:
```

```

def __init__(index, data):
    index.x = data[:]
    index.N = len(data)
    index.stride = 1
    index.i = 0
    index.j = 0
    index.done = False

def tick(index):

    # butterfly
    a = index.x[index.i + index.j]
    b = index.x[index.i + index.j + index.stride]
    index.x[index.i + index.j] = a + b
    index.x[index.i + index.j + index.stride] = a - b

    # passive increment logic
    index.j = bool(xor((index.j + 1), index.stride)) * (index.j + 1)
    # ((a*!b) + (!a*b)) * a

    index.i = bool(xor(index.i, (index.N - 1))) * (index.i + not(index.j) * (index.stride << 1))
    # ((a*!b) + (!a*b)) * (a + !c * d)

    index.stride <= not(index.i)

    index.done = bool(index.stride & (index.N - 1))

```

and when you apply some handy digital design theorems

```

class FWHT:
    def __init__(index, data):
        index.x = data[:]
        index.N = len(data)
        index.stride = 1
        index.i = 0
        index.j = 0
        index.done = False

W = (1 << BIT_WIDTH) - 1 # set this to whatever width.
                        # Typically, BIT_WIDTH = 7 (N up to 128)

def bit_not(x):
    # need this to get past the bitwise
    # inaccurate python "feature"
    return ~x & W

def tick(index):

    # butterfly
    a = index.x[index.i+index.j]
    b = index.x[index.i+index.j+index.stride]
    index.x[index.i+index.j] = a + b
    index.x[index.i+index.j+index.stride] = a - b

    # passive increment logic

    # ((a*!b) + (!a*b)) * a = a(a*!b) + a(!a*b) = a(a*!b) = a * !b

```

```

# a*!a = 0 and a * a = a by the idempotent and complement laws
index.j = (index.j + 1) & bit_not(index.stride)

# unfortunately this can't be reduced lower than using the add;
# we have to use a trick instead
j_wrap = int(not index.j)
index.i = (index.i + (j_wrap*(index.stride<<1))) & (index.N-1)

index.stride <= (not(index.i) * not(index.j))

index.done = bool(index.stride & index.N)

```

already looking more like verilog!

Simulation

```

SD403_TrailCamera_NDSU/sim Sat Feb 07 22:00
on main z python3 wht_sim.py
Walsh-Hadamard Transform Validation

```

```

=====
N=  2 | cycles=   1 | expected=   1 | PASS
N=  4 | cycles=   4 | expected=   5 | PASS
N=  8 | cycles=  12 | expected=  15 | PASS
N= 16 | cycles=  32 | expected=  39 | PASS
N= 32 | cycles=  80 | expected=  95 | PASS
N= 64 | cycles= 192 | expected= 223 | PASS
N=128 | cycles= 448 | expected= 511 | PASS
=====

```

All tests passed.

Demo: N=8

```

Input:  [1, 2, 3, 4, 5, 6, 7, 8]
WHT:    [36, -4, -8, 0, -16, 0, 0, 0]
Inverse: [1, 2, 3, 4, 5, 6, 7, 8]

```